

M&S: Final Assignment

Robot arm with actuation and control

Group 02:

Allard van der Hooft - s2862557

Joost Verhoeven - s2519321

Milou de Zwaan - s2629070

June 21, 2026

Contents

1	Introduction	1
2	Model	2
2.1	Theoretical background	2
2.1.1	Motors and gearboxes	2
2.1.2	3D Mechanics Model	3
2.1.3	Series Elastic Actuator	8
2.2	Model overview	8
2.3	Modelling choices	11
3	Simulation	12
3.1	Tuning of current controller	12
3.2	Tuning torque controller	12
3.3	Tuning of impedance controller	13
3.4	Limit check	14
3.5	Testing gravity compensation	16
3.6	Tracking performance	17
4	Discussion & Conclusion	18
4.1	Discussion	18
4.2	Conclusion	19
	References	19

1 Introduction

For the final assignment of Modelling&Simulation, we integrate all topics learned during the course. This report details the modelling and simulation of a 3D, 3-DOF robotic manipulator designed to feed a user autonomously. Given the proximity to humans, the design prioritizes inherent mechanical safety and spatial precision over operational speed. This makes an impedance-controlled robotic arm with Series Elastic Actuators (SEAs) a great design choice.

First, a mechanical model of the robotic arm will be created in the 20-sim 3D Mechanics Editor. The configuration of the arm will be chosen in such a way that the workspace of the robot is appropriate to the task of feeding a user. The focus here will lie on the robotic arm itself, not the end effector (which should be able to pick up food) or a sensing and/or high-level control system capable of detecting food, the user, and generating appropriate actions.

Then, an impedance controller with gravity compensation will be implemented to control the robotic arm. Here, we will first use ideal effort sources as motors actuating the joints of the robot. This way, we can test if our derived forward kinematics and Jacobian of the 3D model are correct. Also, we can more easily design and accurately test the gravity compensation feedforward part of the controller. Lastly, this will help us get a good initial design choice for the stiffness and damping values to be used in the impedance controller.

After the impedance controller, we will implement the three SEAs. A SEA model will include motor, gearbox, and series elastic element dynamics. Two separate controller loops will be used: a mid-level torque controller and a low-level current controller. Together with the estimate of the delivered torque from the spring deflection, these controllers will regulate the output torque of the SEA and try to follow the reference as provided by the impedance controller as closely as possible.

All systems will first be modelled in continuous time, and afterward, the controllers will be discretized to match a real-world realization. This way, we can rule out problems caused by discretization earlier in the design process. Finally, the model as a whole will be tested, verified, and validated. Controller gains will be fine-tuned based on new dynamics caused by the combination of the separate submodels to achieve acceptable performance. Hardware limits will be checked as well to make sure the components stay within their operating ranges when the robot is performing its task.

2 Model

2.1 Theoretical background

2.1.1 Motors and gearboxes

To help us focus on modelling the robotic arm, the choice was made to base the arm dimensions on a specific motor/gearbox combination, instead of the other way around. This way, we prevented having to do extensive research into the end-effector workspace and instead maximized the workspace for a given drivetrain and chosen margins.

For all three degrees of freedom, we will use the Maxon DC-motor RE30 with a 33:1 two-stage planetary gearbox [1], [2]. The parameters from the catalogues of the motor and gearbox that are relevant to our model can be seen in Table 1. We have used this motor/gearbox combination in earlier assignments, so we know we can make it work and thus focus more on the complete system.

Instead of assuming the motor and gearbox always operate at maximum efficiency, we model the losses as Coulomb and viscous friction. We do this in such a way that the maximum efficiency is obtained at the nominal operating points of the motor and gearbox. For the motor, we have

$$\begin{aligned}d_c &= no_load_current \cdot torque_constant = 3.55 \text{ mNm} \\d_v &= d_c / no_load_speed = 0.00387 \text{ mNm/s/rad}\end{aligned}$$

Table 1: Motor and gearbox parameters

Motor		Gearbox	
Torque constant	52.1 mNm/A	Reduction	33:1
Rotor inertia	34.9 gcm ²	Inertia	0.8 gcm ²
Inductance	0.513 mH		
Resistance	2.45 Ω		
No load current	68.2 mA		
No load speed	8760 rpm		
Nominal speed	8010 rpm		
Nominal torque	87.2 mNm	Max. cont. torque	2.25 Nm
Max. rpm	12000 rpm	Max. input rpm	6000 rpm
Max. torque	87.2 mNm	Max. output torque	3.4 Nm
Max. efficiency	89%	Max. efficiency	75%
Motor nominal voltage	48 V		
Rated motor current	1.74 A		
Stall current	19.6 A		
Weight	260 g	Weight	162 g

To make sure these still combine to obtain the maximum efficiency, we choose to build up the total friction in the model by taking 14% of the Coulomb friction, and the other 86% is taken to be viscous friction to ensure that it follows the real world efficiency from the datasheet.

For the gearbox, we assume the maximum efficiency is achieved while operating at nominal speed, which we assume to be half the maximum speed, and at the maximum continuous torque. At this optimal operating point, we calculate to lose

$$(1 - \text{max_efficiency}) \cdot \text{max_cont_torque} / \text{reduction} = 17.045 \text{ mNm}$$

on the input of the gearbox due to friction. Since this is a planetary gearbox, we assume most friction is Coulomb friction (80%) since most of the friction in the model will be mechanical resistance, so we have $d_c = 0.8 \cdot 17.045 = 13.636 \text{ mNm}$ and $d_v = 0.2 \cdot 17.045 / \text{nominal_speed} = 0.01085 \text{ mNm/s/rad}$.

Combining these mechanical friction losses of the drivetrain gives $d_c = 0.14 \cdot 3.55 + 13.636 = 14.127 \text{ mNm}$ and $d_v = 0.86 \cdot 0.00387 + 0.01085 = 0.01418 \text{ mNm/s/rad}$. In addition to these mechanical losses, we also model electrical losses. These are caused by the resistance of the motor, and we add 0.3Ω to account for the wires connecting the motors.

2.1.2 3D Mechanics Model

To accurately represent the robot's dynamics in the 20-sim simulation environment, the system's characteristics must be established. First, we determine the robotic arm configuration. Then, we specify the characteristics of the system, which are the exact mass, the location of the center of mass (CoM), and the rotational inertia of each physical component. Also, we determine the forward kinematics and Jacobian of our system for later use in the impedance controller, as well as the gravity torques acting on the joints, which we will counter in our gravity compensation feedforward part of the controller. Lastly, we specify the characteristics of the series elastic element.

Kinematic design

We base the design of our robotic arm on our previous assignment. The three joints are chosen to be rotations around the Z-axis and twice around the Y-axis, so a 3-DOF 3R (yaw, pitch, pitch) robot. This creates an arm with a shoulder and elbow that can rotate around its base. This configuration will allow the robot to pick up food situated on the table around it and bring it up to the mouth of the user. See Figure 7 in subsection 2.2 for a visual representation of the robotic arm.

As mentioned before, we will base the arm dimensions (and thus link lengths) on the system dynamics and limits of our actuators. These will thus be determined during the Dynamic Design of the system.

Dynamic design

To determine the dynamics of the robot, the following assumptions were made: the robot's material is PLA with a density of 1240 kg/m^3 . This was done to make it easier to 3D print the design and to test the simulation results at a later stage. All links were made fully solid, even though this may not be the most efficient for the robot's performance and the mass of the links after the shoulder joint. However, this was done to account for the extra weight of wires, sensors, and other components left out in our simplified design. Finally, the mass of the motors and their casings is modeled as a cube to simplify inertia calculations, and the casings are treated as additional 3D-printed masses that could be used for the same physical testing at a later stage. To ensure that the robot's motors can handle the weight of the links, we test the robot in its most horizontally extended position and allow the resulting gravitational torques in the joints (the torques required to keep the arm still) to be a maximum of 1.3 Nm. This leaves torque capacity, which can be used for acceleration/movement of the arm and/or carrying small loads as well as resistance or other modeling elements that were not considered.

As seen in Figure 1, all the known parameters of the system were defined, including the 0.04 m dimensions of the cubic motor casings and the mass equations of the structural links. The radius of the cylinders and cone, as well as the size of the cube, were chosen such that the relative sizes of the different components to one another looked acceptable. This was done to ensure that it would be feasible to 3D print the design at a later stage for physical testing. A critical variable is the length of the cylindrical links, denoted as the unknown L , which dictates both the mass and moment arm of the system as detailed in Table 2.

The cylinder height L was calculated by setting the 1.3 Nm limit equal to the sum of the individual torques generated by the links after the elbow using figure 1 and the mass formulas from table 2. To do this the following formula was used

$$\tau = m \cdot g \cdot r \cdot \sin(\theta)$$

where r is the distance to the center of mass of the different links in accordance to their mass m . Because the weight of the structural links and the distance to their centers of mass both scale with the unknown length L , multiplying these two variables to calculate torque generates L^2 terms. As a result, summing the torques for the entire arm simplifies the system into a single quadratic equation. Solving this equilibrium calculation yields a conclusive maximum safe link length of $L = 0.106 \text{ m}$. For this model and the resulting calculation, it was assumed that both structural cylinders and the total distances between the motors maintain a strict 1:1 ratio.

Table 2: Mass calculations and system parameters based on PLA density (1240 kg/m³).

Component	Formula	Calculated value
Motor mass	motor + gearbox + casing mass	0.461 kg
Cylinder link	$(0.02^2 \cdot \pi) \cdot 1240 \cdot L$	$1.558 L$ kg
Cube	$(0.04 \cdot 0.04) \cdot 0.04 \cdot 1240$	0.07936 kg
End-effector (cone)	$\frac{\pi \cdot 0.02^2}{3} \cdot 0.04 \cdot 1240$	0.0208 kg
Max horizontal torque	percentage of max. drivetrain torque	1.3 Nm
Gravity (g)	—	9.81 m/s ²

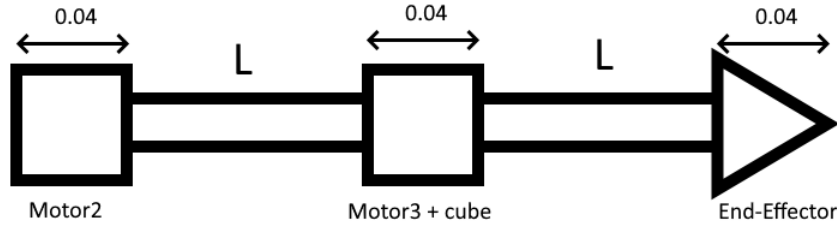


Figure 1: Side-drawn non-perspective diagram to find the length of the cylinder.

With the missing length of the cylinder links known, all the dimensions and weights of the system are known. The final step required to construct the 3D mechanical model in 20-sim accurately is determining the moments of inertia (I_x, I_y, I_z) for each component. This is achieved by applying standard inertia tensor formulas for the respective 3D geometric shapes evaluated at their centers of mass (CoM).

For the cube motor with side lengths of $a = 0.04$ m and mass $m_{\text{Motor mass}}$. For a cube, the mass is distributed evenly across all three axes, which means the inertia is symmetrical, which gives:

$$I_{xx} = I_{yy} = I_{zz} = \frac{1}{6} m_{\text{Motor mass}} a^2$$

For the solid cylinder links with a radius of $r = 0.02$ m, length of $L = 0.134$ m, and mass $m_{\text{Cylinder link}}$. For a cylinder, the mass is distributed symmetrically around its longitudinal z-axis but extended along its length, which means the inertias are equal but differ from the longitudinal inertia, which gives:

$$I_{xx} = I_{yy} = \frac{1}{12} m_{\text{cyl}} (3r^2 + L^2)$$

$$I_{zz} = \frac{1}{2} m_{\text{cyl}} r^2$$

For the conical end-effector with a base radius of $r = 0.02$ m, height of $h = 0.04$ m, and mass $m_{\text{End-effector}}$. For a cone, the mass is rotationally symmetric around its central z-axis but tapers

along its height, which means the inertias are equal but differ from the longitudinal inertia, which gives:

$$I_{xx} = I_{yy} = \frac{3}{80} m_{ee} (4r^2 + h^2)$$

$$I_{zz} = \frac{3}{10} m_{ee} r^2$$

By calculating all the proper, realistic weights and inertias and entering them into the 3D mechanics blocks in 20-sim, the simulation will accurately represent the system's mechanics.

Forward kinematics

The position of the tip of the end-effector can be straightforwardly derived:

$$p_e = \begin{pmatrix} L_2 \cos(q_1) \sin(q_2) + L_3 \cos(q_1) \sin(q_2 + q_3) \\ L_2 \sin(q_1) \sin(q_2) + L_3 \sin(q_1) \sin(q_2 + q_3) \\ L_0 + L_1 + L_2 \cos(q_2) + L_3 \cos(q_2 + q_3) \end{pmatrix}$$

where q_1, q_2, q_3 are the joint angles, and $L_0 \dots L_3$ are the vertical distance from the ground to the Z-joint, the vertical distance from the Z-joint to the shoulder, the distance from the shoulder to the elbow, and the distance from the elbow to the tip of the end-effector, respectively.

Jacobian derivation

The geometric Jacobian is formed by stacking the unit twists next to each other:

$$J(q) = [\hat{T}_1^{0,0}(q) \hat{T}_2^{0,0}(q) \dots \hat{T}_n^{0,0}(q)] \quad (1)$$

To achieve this, the local twists must be expressed in the base frame by multiplying the adjoint of the direct kinematics with the local twist:

$$\hat{T}_i^{0,0}(q) = Ad_{H_{i-1}^0(q)} \hat{T}_i^{(i-1),(i-1)} \quad (2)$$

The adjoint matrix can be calculated using the following formula:

$$Ad_{R_i^2} := \begin{pmatrix} R_i^1 & 0 \\ \bar{p}_i^2 R_i^2 & R_i^2 \end{pmatrix} \quad (3)$$

Using Brockett's formula for direct kinematics, the transformation matrix $H_n^0(q)$ can be solved:

$$H_n^0(q_1, q_2, \dots, q_n) = \frac{e^{\bar{T}_1^{0,0} q_1} H_1^0(0)}{H_1^0(q_1)} \frac{e^{\bar{T}_2^{1,1} q_2} H_2^1(0)}{H_2^1(q_2)} \dots \frac{e^{\bar{T}_n^{(n-1),(n-1)} q_n} H_n^{n-1}(0)}{H_n^{n-1}(q_n)} \quad (4)$$

In our case, we have the local twists (Yaw-Pitch-Pitch):

$$T_1^{0,0} = \begin{pmatrix} 0 \\ 0 \\ 1 \\ 0 \\ 0 \\ 0 \end{pmatrix}, \quad T_2^{1,1} = \begin{pmatrix} 0 \\ 1 \\ 0 \\ 0 \\ 0 \\ 0 \end{pmatrix}, \quad T_3^{2,2} = \begin{pmatrix} 0 \\ 1 \\ 0 \\ 0 \\ 0 \\ 0 \end{pmatrix}$$

We take the reference configuration to be upright along Z:

$$H_1^0(0) = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & L_1 + L_2 \\ 0 & 0 & 0 & 1 \end{pmatrix}, \quad H_2^1(0) = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & L_3 \\ 0 & 0 & 0 & 1 \end{pmatrix}, \quad H_3^2(0) = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & L_4 \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

This results in a Jacobian with linear velocity part (J_v)

$$J_v = \begin{pmatrix} -\sin(q_1)(L_4 \sin(q_2 + q_3) + L_3 \sin(q_2)) & \cos(q_1)(L_4 \cos(q_2 + q_3) + L_3 \cos(q_2)) & L_4 \cos(q_2 + q_3) \cos(q_1) \\ \cos(q_1)(L_4 \sin(q_2 + q_3) + L_3 \sin(q_2)) & \sin(q_1)(L_4 \cos(q_2 + q_3) + L_3 \cos(q_2)) & L_4 \cos(q_2 + q_3) \sin(q_1) \\ 0 & -L_4 \sin(q_2 + q_3) - L_3 \sin(q_2) & -L_4 \sin(q_2 + q_3) \end{pmatrix}$$

and angular velocity part (J_ω)

$$J_\omega = \begin{pmatrix} 0 & -\sin(q_1) & -\sin(q_1) \\ 0 & \cos(q_1) & \cos(q_1) \\ 1 & 0 & 0 \end{pmatrix}$$

Gravity compensation

To accurately compensate for the effects of gravity on the robot arm, the masses of the different parts of each link are lumped together (see Figure 2), and the CoM location of the link is determined. Since the first link is always vertical, the first joint will never feel gravity, and thus this link is skipped. For the next two links, the cylinder part weighs $1.558L = 1.558 \cdot 0.106 = 0.1651$ kg. The link masses and the distance to the CoM from the previous joint are then:

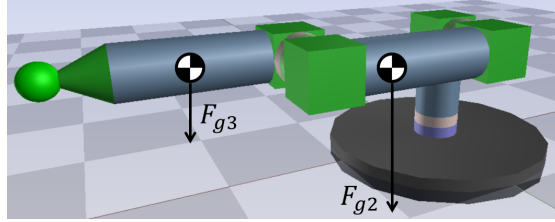


Figure 2: Gravity compensation derivation visualization.

$$m_2 = m_{cube} + m_{cylinder} + m_{motor} = 0.07936 + 0.1651 + 0.461 = 0.70546 \text{ kg}$$

$$m_3 = m_{cube} + m_{cylinder} + m_{cone} = 0.07936 + 0.1651 + 0.0208 = 0.26526 \text{ kg}$$

$$r_2 = (r_{cube}m_{cube} + r_{cylinder}m_{cylinder} + r_{motor}m_{motor})/m_2 \\ = (0 \cdot 0.07936 + 0.073 \cdot 0.1651 + 0.146 \cdot 0.461)/0.70546 = 0.1125 \text{ m}$$

$$r_3 = (r_{cube}m_{cube} + r_{cylinder}m_{cylinder} + r_{cone}m_{cone})/m_3 \\ = (0 \cdot 0.07936 + 0.073 \cdot 0.1651 + 0.146 \cdot 0.0208)/0.26526 = 0.05688 \text{ m}$$

We can now determine the joint torques generated by gravity to be the following:

$$\tau_g = \begin{bmatrix} 0 \\ m_2 g r_2 \sin(q_2) + m_3 g (L_2 \sin(q_2) + r_3 \sin(q_2 + q_3)) \\ m_3 g r_3 \sin(q_2 + q_3) \end{bmatrix} \quad (5)$$

where $L_2 = 0.146$ m is the total length of link 2 (from joint 2 to joint 3), g is the gravitational acceleration constant, and q_2 and q_3 are the angles of joint 2 and 3, respectively. We can now compensate for this gravity by adding the negated gravity torques via a feedforward part in the controller.

2.1.3 Series Elastic Actuator

To ensure safe human-robot interaction during robot operation, series elastic actuators are integrated into the joints to provide passive mechanical compliance. Balancing this inherent safety with the precision required for the robot’s operation dictates careful tuning of the joint stiffness (k). To find this spring value, we use the following method seen in the bond graph Figure 3. The method is to place a 0-flow source after the motor effort source, setting the velocity there to 0 (meaning the drive shaft is locked), so only the robot’s gravity is working. To be more precise, we use a 0-junction to subtract the gravity-induced velocity from the motor velocity, so it would be possible to tune k with the motor velocity. In addition, a resistor is used to prevent it from oscillating indefinitely. We can now enter a k value, plot the robot’s angle, subtract the initial angle from the final angle, and find the total sag based on gravity. For the base yaw (Z_1) and elbow pitch (Y_2) joints, the stiffness was optimized to 15 Nm/rad. This value restricts the gravitational deflection to approximately 0.53° on the elbow. This constrains the end-effector deviation to within 3 mm. Conversely, the shoulder pitch joint (Y_1) requires a stiffness strictly greater than 15 Nm/rad. Because Y_1 must counteract the combined gravitational torque of links 2 and 3 and also the payload, a significantly higher spring constant is necessary to prevent excessive deflection while still preserving the compliance required for impact mitigation. So, for Y_1 we use $k = 30$ Nm/rad which gives a sag of 2.5 degrees.

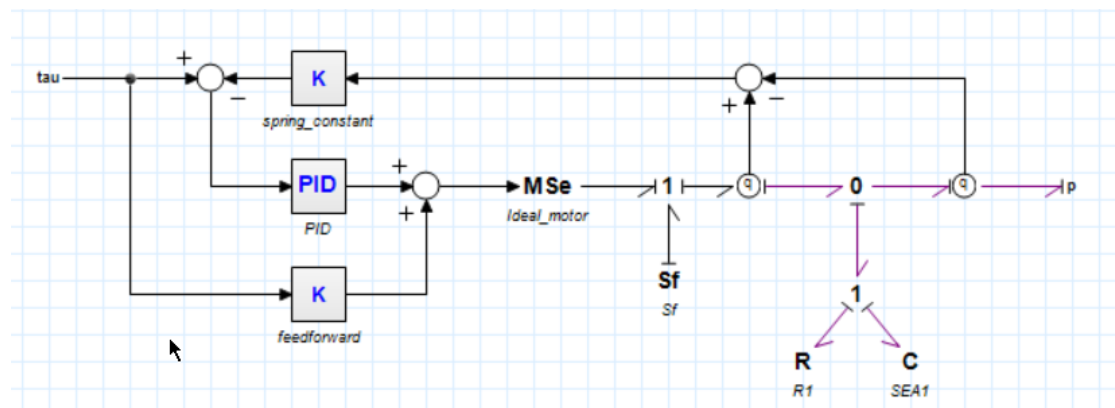


Figure 3: Bond graph to find spring value (k).

2.2 Model overview

The 20-sim bond graph model is built up of three main bond graph areas as seen in Figures 4, 5, 6, and 7. Figure 4 shows the broad overview of the model with a ref block inputting a reference position to the (impedance) controller block that, in turn, outputs the reference torques that the motors need to apply to the robot. The three motors in each joint then apply an actual torque to the 3D mechanics robot arm, which results in joint motion/velocity. Finally, the robot arm sends back the three joint angles to the controller, which the controller uses for gravity compensation

and impedance control. Even though it is not clearly visible in the figure, the links between the motors and the 3D model are indeed power bonds with a torque and angular velocity pair.

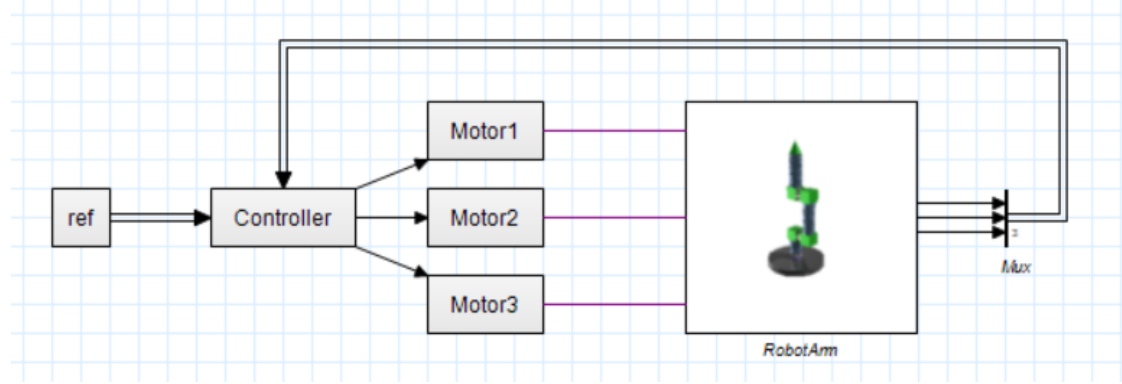


Figure 4: Bond graph model of the main model architecture.

The controller is the brain of the system; as noted above, it calculates the reference torques the actuators must deliver. It does this using a couple of steps as seen in Figure 5. First, the current joint angles (q) are sampled and fed into the gravity compensation block to compute the exact counter-torque required to prevent the physical links from sagging under their own weight. Simultaneously, these joint angles are used to determine the end-effector position, which in turn is used in combination with the sampled desired target positions (reference) and processed by the impedance controller to compute the compliant torques required to move the arm to its target position safely, i.e., with the chosen impedance stiffness and damping. To this end, it uses the Jacobian (J) with the current joint angles. These two results are then summed to obtain the required total 3D torque vector. Finally, a splitter breaks this combined vector into the three individual reference torques (τ_1 , τ_2 , and τ_3) that are sent to the individual actuators for each joint.

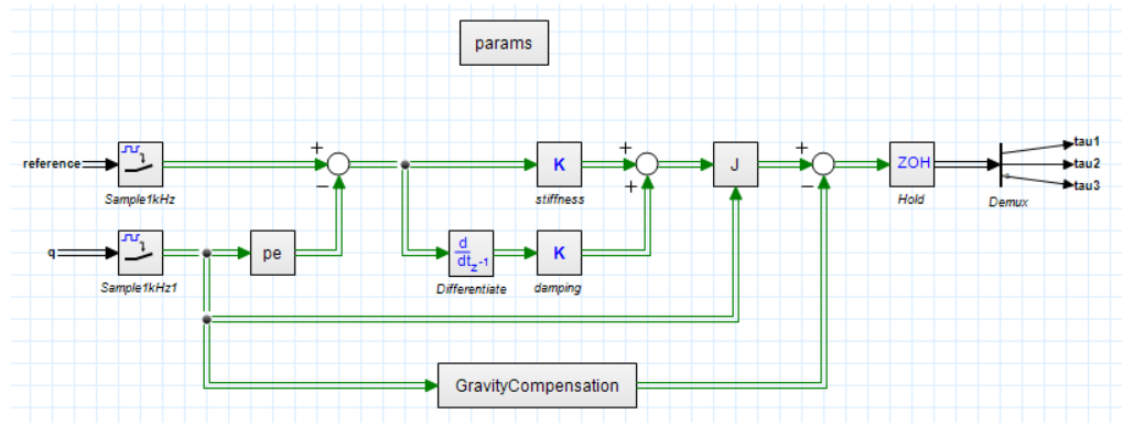
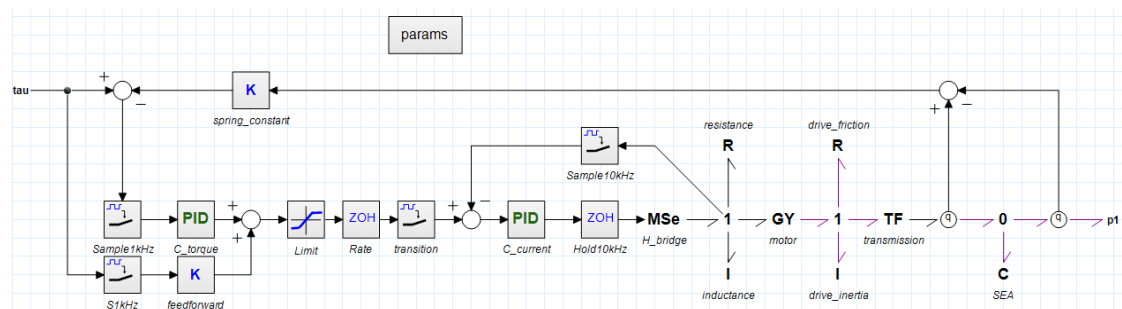


Figure 5: Bond graph model of the controller

The motor submodel acts as the actuator of the system, translating the controller's electrical

signals into physical movement. As shown in Figure 6, the physical part of the submodel operates by using the required electrical voltage (u) generated by the low-level current controller as its primary input. This required electrical voltage is perfectly transformed into actual electric voltage and is then transformed into mechanical torque, after the effects of the motor resistance and inductance are taken into account. The mechanical torque on the motor output shaft experiences drivetrain friction, as explained in subsection 2.1, as well as drivetrain inertia, which consists of a combination of the motor and gearbox inertia. The torque is then scaled by the gearbox ratio before being transferred into the series elastic element. This spring is modeled by a C element, which stores energy based on the deflection of the spring. Two q sensors, one before and one after the spring, measure the deflection of the spring, which is transformed into the related spring torque through the spring constant.



To simulate the physical structure of the robotic arm, we built a mechanical model using the 20-sim 3D Mechanics Editor, as shown in Figure 7. This submodel acts as a simulation of the actual physical plant for our system. To ensure the simulation behaves exactly like the real prototype for later physical testing, we used the masses, dimensions, and rotational inertias we calculated earlier and entered them directly into the corresponding rigid body blocks. This guarantees that the simulated gravitational pull and the resulting joint dynamics accurately match the arm’s physical design.

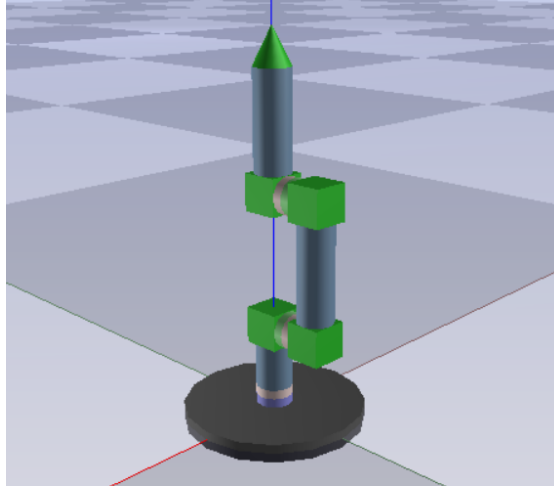


Figure 7: 3D mechanics physical model

2.3 Modelling choices

While modelling the robotic arm, some choices were made that need to be named and/or explained. These choices concern joint friction, joint angle sensing, collision detection, discretization, and noise/disturbances additions.

In the 3D model of the robotic arm, no friction was implemented; all friction is modelled to be inside the actuators. Thus, without actuation, no energy is lost, and the robotic arm swings indefinitely. This choice was made to more easily keep track of friction and energy losses in the model and simplify the design (process).

Secondly, we modelled the robotic arm as having absolute encoders in its joints. Thus, the 3D model outputs its exact joint configuration, which is used by the impedance controller. It even does so continuously, although we sample the signal inside the controller for realism. Even more realistically, though, would be to have the sensor incorporated in the actuator, as most real-life robotic arms have, which could also be a velocity sensor that is integrated to get the joint angles. However, this would add complexity, and since having absolute encoders in the joints is possible, we chose to go this way.

Third, no collision detection was implemented in the model. This is most notable through the robot arm being able to move through itself. To be more precise, dynamic joint limits would have to be imposed to prevent the robot from damaging itself or from obtaining unrealistic simulation results. However, this was not the focus of this assignment, so we chose to keep the model simpler and ignore collision physics.

To discretize the model, signals are sampled at specific frequencies and converted back into continuous time through zero-order-holds. As can be seen in our Model overview, this is sometimes done in a somewhat cumbersome way. The reference is, for example, implemented in continuous time, and then sampled in the impedance controller, while it could have been implemented discretely straight away. Also, between the impedance controller and the motors, the reference torques are made continuous and subsequently discretized again. This is purely a result of the longer design process, but since it ultimately does not affect the system, it was not changed. For the sampling frequencies, we chose 1 kHz for the impedance controller and

torque controllers, and 10 kHz for the current controllers. Although 1 kHz might be a bit high for the impedance controller, these values are quite realistic and chosen so as not to have too much difficulty with the discretization affecting the system performance.

Lastly, no noise or disturbances were added, and no quantization. This means that all signals in the model were exact (up to numerical precision). In reality, all digital signals would not only be discrete but also quantized, leading to small errors. Noise in the sensors and actuators would also increase system errors and possibly disturb the controller's function. We, however, argue that the effect of quantization would be small/neglectable for our simulation. Noise could significantly affect the system, but it was chosen to be outside the scope of our report. Disturbance-wise, we do plan to test adding a load to the end-effector, which could be viewed as a disturbance. However, additional disturbances of different types might have interesting and important effects on the system, but this was also chosen to be outside our scope.

3 Simulation

3.1 Tuning of current controller

The current control section of our model was put into a new 20-sim model. The controller was tuned until we were satisfied with its performance. It reached the reference in about 0.1s. The reference was set to 1, if we look at the current demand in the full model this was 20A for a short time (because large error at starting pose) after this, for following the trajectory it was around 1A. Further testing revealed that the system saturates at approximately 17A. Therefore, tuning at the 1A operating point seems like a good value.

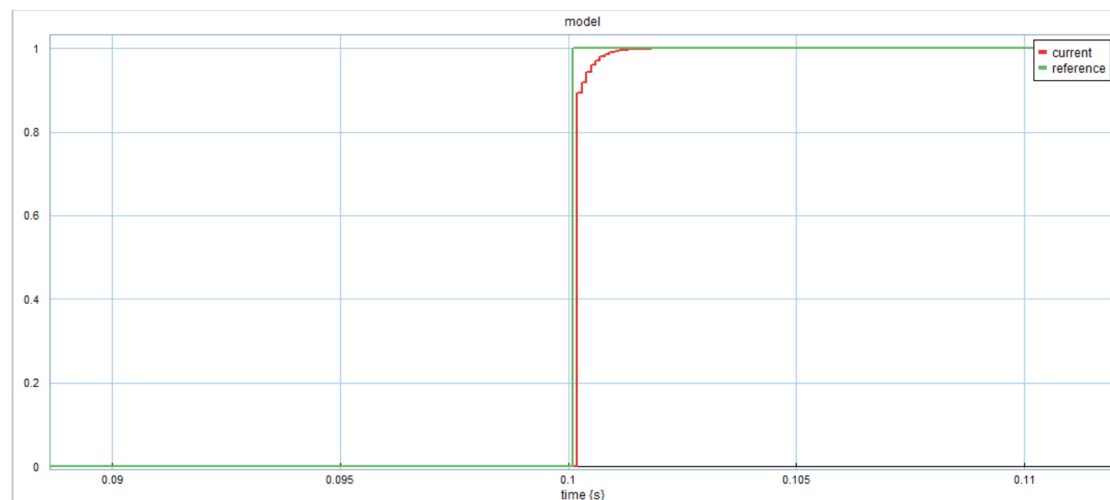


Figure 8: Current controller, red: reference, green: measured current

3.2 Tuning torque controller

The torque controller components were added to a new 20-sim file. The robot arm was initially locked by applying a zero-flow source through a 1-junction. This ensures that the controller is tuned solely based on the compression of the Series Elastic Actuator (SEA), without the spring

(de)compressing due to arm movement. Later, in the complete model, this lock was removed, and the controllers were fine-tuned for the change in dynamics.

In the torque feedforward, we initially attempted to account for mechanical friction by including the nominal motor and gearbox efficiencies (89% and 75%) directly in the constant feedforward calculation. However, doing so caused the feedforward to overcompensate aggressively, which resulted from tests with the PID feedback controller disabled (see Figure 9). Therefore, we chose to drop the efficiency terms and only use a constant based on the motor torque and gearbox ratio ($k = 52.1 \cdot 33 \text{ mNm/A}$). This value was chosen because it provides an estimated baseline for the motor current required to produce a given amount of torque. When the efficiency terms were removed, it seemed to behave more as expected (Figure 10).

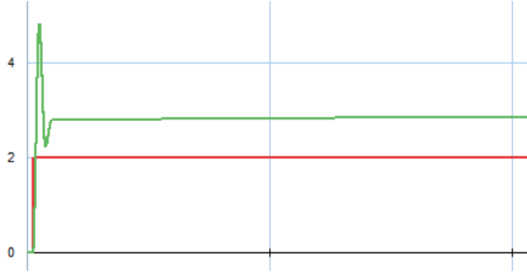


Figure 9: Feedforward with efficiency terms, red: reference, green: measured torque

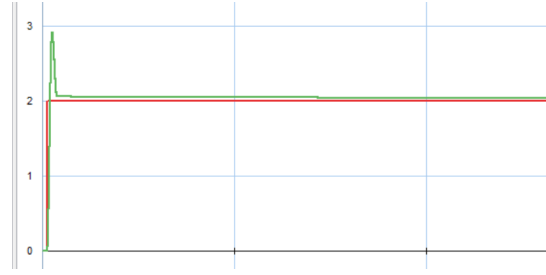


Figure 10: Feedforward without efficiency terms, red: reference, green: measured torque

After verifying the feedforward, the PID feedback controller was enabled and tuned to create a slightly faster and more robust reaction. See Figure 11.

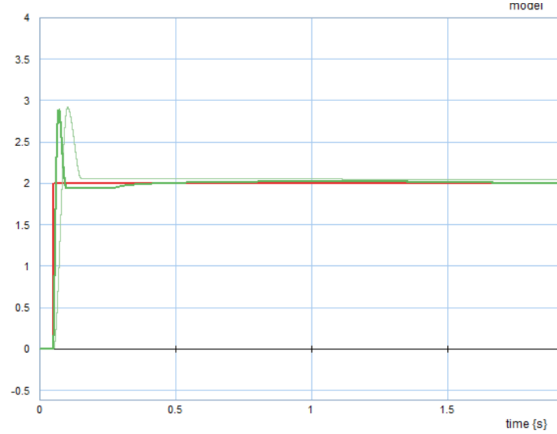


Figure 11: red: reference, green: response with pid, lightgreen: without pid

3.3 Tuning of impedance controller

The impedance controller was tuned, with a spring stiffness high enough that it does not exceed the motor/gearbox limits, while still reacting somewhat fast. Then, damping was added to remove oscillations. The performance of the controller can be observed in Figure 20.

3.4 Limit check

This was to verify that the robot does not exceed limits in the datasheet.

Motor current At the start, when the position error is large, the current will spike to max around 18.9 A, as shown in Figure 12. This is quite high, although not reaching the stall current of 19.6 A. Additionally, this only happens for a very short time, and the oscillation dies out in 0.1 s. Then it operates between a boundary of ± 0.4 A, which is below the nominal current of 1.74.

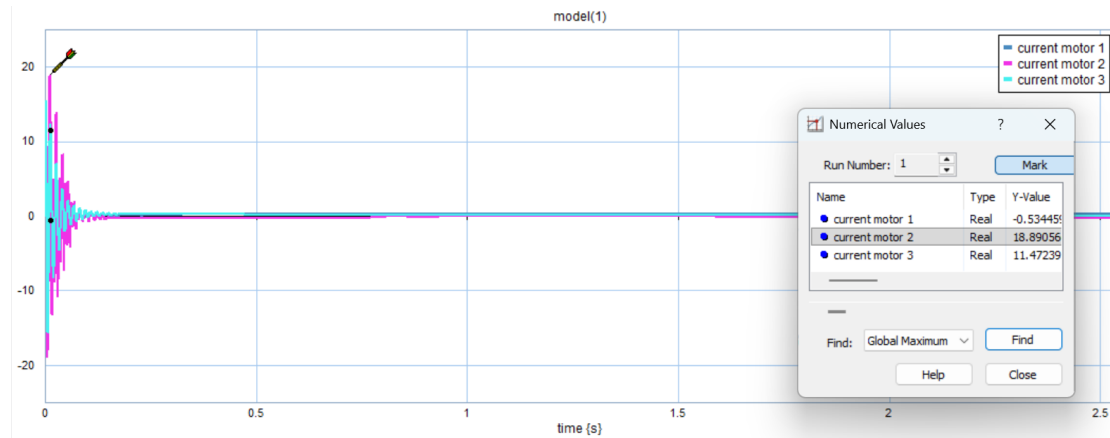


Figure 12: motor current

Motor voltage The max voltage in the motor data sheet is 48V, as shown in figure 13 below, the robot does not exceed this. This is because we have limited the max voltage in the current controller.

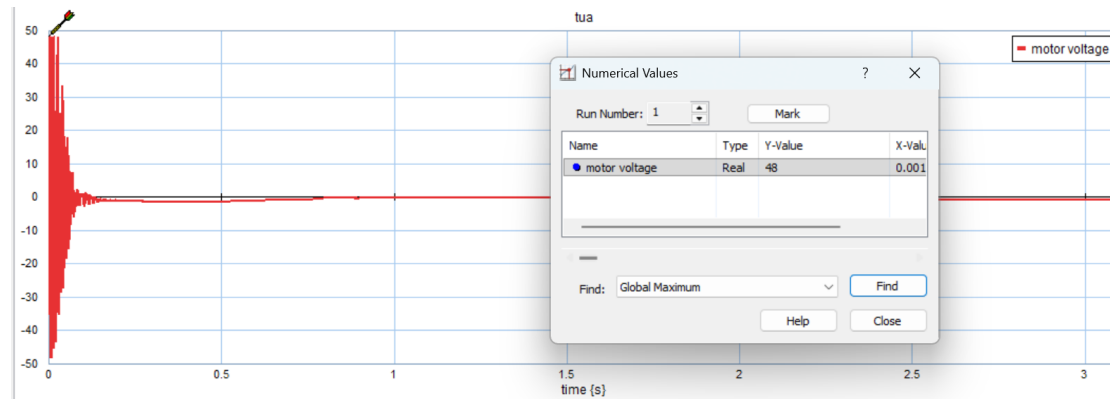


Figure 13: Motor Voltage

Motor torque The output torque of the motor was plotted; the results are shown in Figure (14). Because link 2 has to work the hardest, this is the motor that is plotted. During the initial phase, the torque demanded is 0.984 Nm. Like with the current, this spike exists only for a short

moment, and it is still below the max torque (1.020 Nm). After this spike, the normal operation is around 0.022 Nm, which is below the acceptable continuous torque (0.0872 Nm).

Gearbox torque Shown in Figure 14, the gearbox output torque peaks at 1.95 Nm during the initial response, which is still below the datasheet's 3.4 Nm limit. After this spike, the torque is approximately ± 0.3 Nm, which is well below the 2.25 Nm continuous rating.

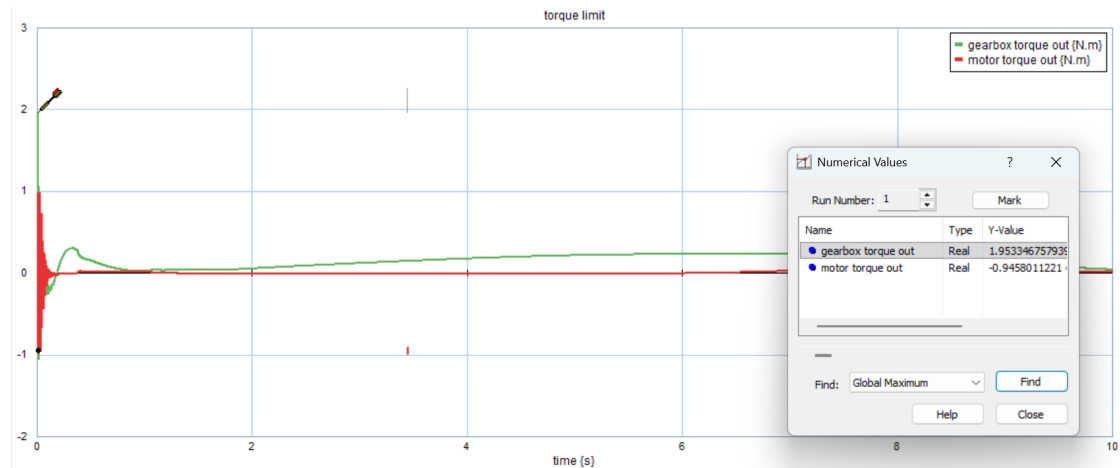


Figure 14: Motor and gearbox torque

Motor and gearbox rpm In the data sheet of the motor, the max RPM and nominal RPM can be found (12000 rpm and 8010 rpm, respectively). To see if the motor stays below this limit, the flow out of the motor shaft is plotted. In Figure 15a maximum of, it can be observed that the max rpm that is reached is 620 rad/s (5920 RPM). So this stays well below the limit. After this spike, it operates at max 11 rad/s (105 RPM), which is below the nominal speed.

For the gearbox, the same approach is taken; the flow into the gearbox is plotted in Figure 16. The max input rpm for the gearbox is 6000 rpm. In the figure, it can be observed that the maximum rpm reached is 620 RPM.

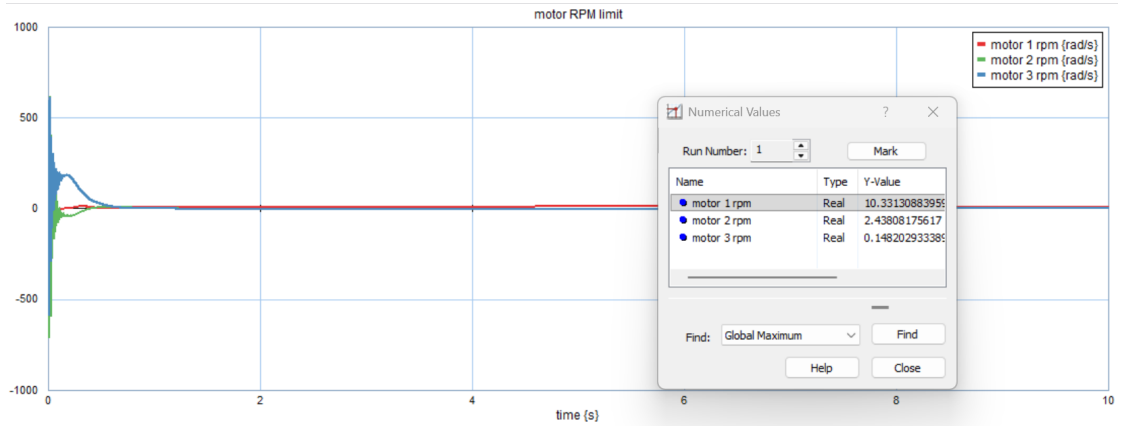


Figure 15: rpm of the motor

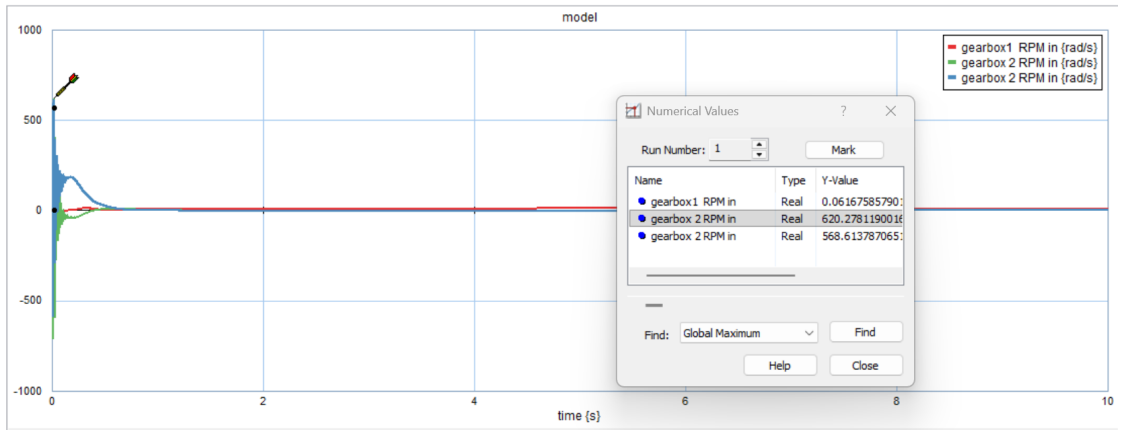


Figure 16: gearbox RPM in

3.5 Testing gravity compensation

To test the gravity compensation, the impedance controller is turned off, and the robot is set to the most horizontally extended position (link 2's initial angle to 90 degrees), see Figure 18. The vertical position of the end effector is plotted (fig 17) to see if, and for how long, this stays constant. The blue line is the z coordinate, which we are interested in. Initially, it can be observed that it drops a bit, and then it stays constant. However, in the model without the discrete controllers, with perfect Mse as motors, it does work perfectly, as shown in Figure 19. This difference could come from the compliance of the sea.

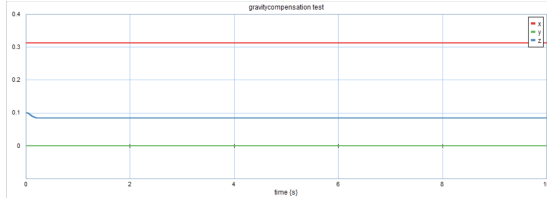


Figure 17: Gravity compensation test

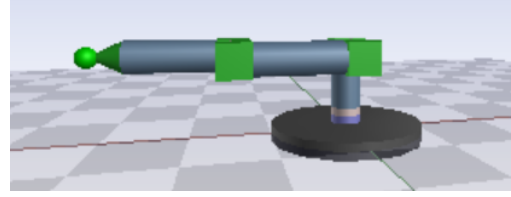


Figure 18: Pose for gravity testing

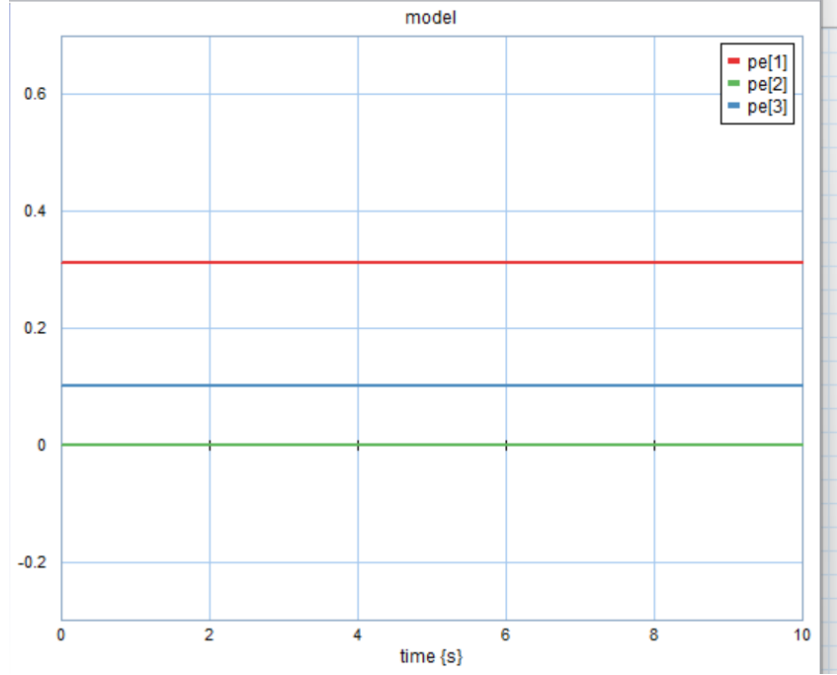


Figure 19: gravity compensation perfect motors, no discrete controllers

3.6 Tracking performance

Finally, we will show the tracking performance with and without load.

In Figure 20, it can be observed that the robot follows the trajectory quite well but not perfectly, so

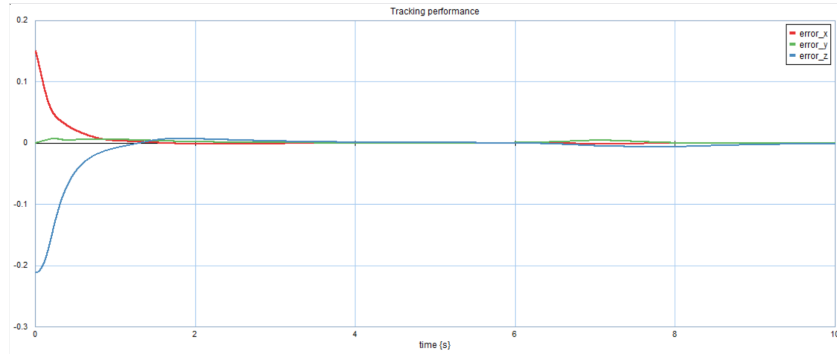


Figure 20: Position error of tracking, without load attached

To demonstrate the robot's interaction with its environment and highlight the effectiveness of our chosen impedance parameters. The following tracking experiment was performed in which the robot acted on an external load (picking up a 100-gram apple). As seen in 21, the arm still follows the general trajectory, but a steady-state error is introduced, causing the end-effector to sag slightly below the vertical Z-setpoint. This behavior perfectly illustrates the intended behavior of our impedance controller. Because the gravity compensator does not account for the apple's weight, this weight is added to the system as a disturbance.

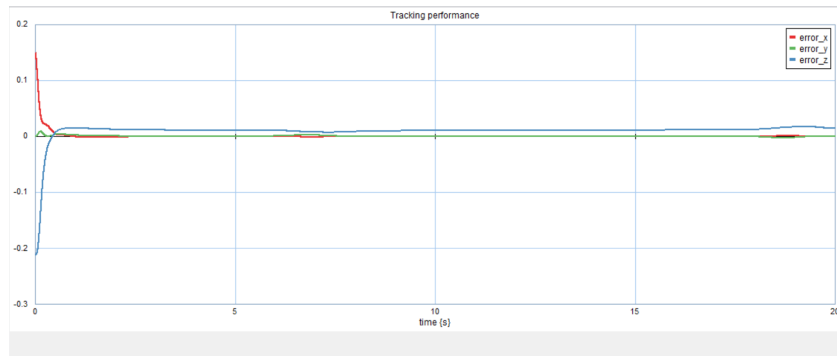


Figure 21: Position error of tracking, with load attached

4 Discussion & Conclusion

4.1 Discussion

Overall, the 3D mechanics and gravity compensation worked well. By modeling the links as solid PLA, the feedforward kept the arm steady even when fully extended. The current controller also performed as expected, settling quickly and staying within continuous limits.

A few issues did come up. The torque feedforward initially overcompensated until we removed the drivetrain efficiency terms. Also, because the simulation starts with a large position error, there is an aggressive initial spike in current (18.8 A) and gearbox torque (1.27 Nm). While technically below the hardware limits, a physical prototype would need a softer start or a trajectory generator to avoid damage. Finally, modeling the links as completely solid PLA made the arm very heavy,

forcing us to limit the link length (L) to 10.6 cm to stay under our 1.3 Nm holding torque limit. For a real version, making the distal links hollow would be better.

Tuning the controllers systematically and effectively also turned out to be a great challenge. We tried a circular tuning scheme, where we tuned an isolated controller, added and tuned an additional (higher-level) controller, and then fine-tuned the lower-level controller. There was, however, quite a lot of influence from one controller on the performance of the other, also between the different motors through reaction forces in the joints and links. This made it difficult to find good controller gains. In the end, we did manage to find a set of gains that resulted in acceptable performance, but additional tuning or a different controller setup might increase the performance further.

Additionally, changing the order of the design process would be beneficial for the development of an actual robotic arm. So, researching the required robot dimensions, determining the link thickness based on maximum load and material stress, calculating the link masses, and then picking a motor and gearbox combination with sufficient torque and speed output. This would result in a drivetrain more suitable for our specific application, and thus also most definitely better performance.

Also, as mentioned in subsection 2.3, no noise or additional disturbances were incorporated in the model. Their effect would, however, be very important for further design and production of the system, and should thus be included in further research and development.

4.2 Conclusion

Allard: For me, the biggest thing I learned is a basic intuition of how systems work and how to model it based on this knowledge by implementing physics/math that I learned during the robotics master and courses throughout my study career.

Joost: It was very nice to see different skills learned during my studies come together into one course, tying them together. It was very educational to design, model, control, and simulate a robotic system at the same time, and it helped me understand the influence these different domains have on each other. It was also very nice to use the knowledge from MDK in a practical application. And although I did not follow the course ESD and struggled a lot with 20-sim, I feel like I learned a lot about Bond graphs and the interaction between the different power domains. I am confident this knowledge will be of aid in the future.

Milou: I liked learning more about modeling, it cleared some things up for me about deriving a jacobian. I liked that we learned about a lot of domains, electrical, mechanical and even temperature. I liked that we also learned more about modeling in 3D because for most courses I had it was limited to 2D

References

- [1] Maxon. “Re 30 ø30 mm, graphite brushes, 60 watt,” Accessed: Jun. 21, 2026. [Online]. Available: <https://www.maxongroup.com/maxon/view/product/268216>.
- [2] Maxon. “Planetary gearhead gp 32 a ø32 mm, 0.75 - 4.5 nm, metal version,” Accessed: Jun. 21, 2026. [Online]. Available: <https://www.maxongroup.com/maxon/view/product/166163>.